Sniffer Dog / AIP: Technical Brief

Independent runtime observation and accountability for autonomous AI in financial systems

VANIAN Technologies | Prepared for the Reserve Bank of India, FinTech Department | September 2026, Revision 11

1. The question this system answers

When an AI agent executes a consequential action inside a regulated entity, today the only account of what happened is the agent's own: its logs, its explanation, its self-report. Sniffer Dog exists to produce a second, independent account, and to compare the two.

For every action it answers: what was authorised (signed before the act by an authority that is not the agent, naming the action, the parameters and the institutional resource), what actually executed (recorded from the kernel by an observer that is not the agent and does not trust the agent), what the institution's own systems received (countersigned by the tool gateway in front of them), and whether they match. Every record is signed and can be verified offline by the institution's auditor or by the supervisor, using keys the institution holds. No step depends on trusting VANIAN.

The design principle that governs everything below: absence of evidence is never converted into an accusation. When the observer was not watching, or was interfered with, or no gateway receipt exists, the system says so and stops. It does not guess.

What changed since Revision 6. The corroboration of Revision 6 established that AIP's *observations* survive comparison with observers it did not author. A second and distinct question is whether AIP's **identity binding** -- the claim that a given process is running given code -- survives the same test. It has now been run against **SPIFFE/SPIRE**, the CNCF-graduated workload-identity standard, end to end: the two authorities agree on a live workload, and when the identity registry is re-pointed at different code the running system is caught diverging from the approved record. Subsection 10b is new and gives the numbers, the soundness control, and the two defects the exercise found *in our own code*. One correction is also recorded there: a claim we had considered making about a published weakness in SPIRE was checked against its primary source and **withdrawn before it was ever written into this brief**. **All of it ran on a development virtual machine; B8 is unchanged.**

What changed since Revision 5. The remaining phases of the hostile-validation programme have been executed, and four things moved. **First, the agent on the other end of the observer is no longer one of ours.** An unmodified third-party agent framework -- LangGraph 1.2.11 -- ran a graph whose node spawned a child process that wrote a file and opened a TCP connection; 167 process exec and fork events from that run reached the collector-signed archive and were verified offline from the published key alone. **Second, the tool boundary of section 6 now ships a working reference gateway rather than a format.** It countersigns every MCP *tools/call* with a gateway-signed receipt over its own recomputation of the argument commitment, and a mis-parameterised call is caught against the decision point's sealed digest *with the agent's own account absent from the evidence*. **Third, delegation-chain attribution is demonstrated on that same real evidence**: the parent-to-child chain is derived only after the archive re-verifies under the published anchor, and the engine is tested to return nothing -- never a guessed parent -- when the fork record is missing. **Fourth, coverage is now stated against an external corpus we did not write**: Atomic Red Team, mapped to MITRE ATT&CK and published as a Navigator layer in which **6 of 20 techniques are marked not covered**. **Fifth, and the substantive one: AIP's observations have been corroborated by two runtime observers we did not build** -- bpftrace and auditd, the latter reaching the kernel by a different mechanism -- watching the same kernel over the same window; neither recorded an execution absent from AIP's signed evidence. Subsection 10a gives the numbers and the two defects the exercise found in our own comparison harness. **All of it ran on a development virtual machine; B8, the run on the institution's own hardware, is unchanged.**

What changed since Revision 4. The Phase 1 exit gate is met end to end on a development host. Its third clause -- *the verifier accepts the batch offline* -- was previously unreachable for a reason no test in the repository could have caught: the collector attested each batch, returned its position in the chain as an acknowledgement, and then **discarded the batch**. The chain position proved a batch had existed; the signed evidence itself did not survive. The collector now keeps every attested batch, publishes its evidence **public** key so an auditor can be given the anchor without anyone unwrapping the private half, and a separate verifier reads the archives with no network, no service and nothing from us. Two further items Revision 4 listed as not demonstrated have also moved: the collector's clock discipline is now inside the signed bytes, and the signing and verification path has been measured, with the result that attestation is bound by storage rather than by cryptography. Sections 10 and 12. **It ran on a development virtual machine; B8, the run on the institution's own hardware, is unchanged.**

What changed since Revision 3. The foreign-eBPF gate attack, which Revision 3 listed as not yet run against the daemon pair, has been executed end to end against both real daemons with the collector signing **IntegrityUnknown** off-host; see section 10. The run was on a development virtual machine, so the institution-hardware gate (B8) is unchanged.

What changed since Revision 2. The probe-to-collector link is now confidential as well as authenticated; the pilot probe drives the file and wire sensors that were previously demonstrator-only; the observer records container placement

independently rather than taking the authorising side's word for it; signing keys can live in a PKCS#11 module instead of a wrapped file; log heads are countersigned by an external witness; the evidence store's controls are backed by an exercised restore; and the adversarial ledger is reproducible by a single command. Seven of the eleven pre-production blockers in section 13 have moved. The hard gate that has not moved is the run on the institution's own hardware, and it is stated as plainly here as it was before.

2. Independent authorities

The security argument rests on separation. Each component holds its own signing key, generated where it runs, and none can produce another's artefact. The first three form the accountability core; the rest attach to it without being able to move a verdict.

Authority	Produces	Key custody	Cannot do
Decision point	Signed *ExpectedExecution* before the act, naming action, parameter commitment and institutional resource; and coverage statements from its own ledger.	Its own key, separate from the agent's.	Cannot observe. Cannot issue a decision into a window it has already declared closed.
Observer / collector	Signed *AttestedBatch* of kernel-recorded observations, sequence and chain head, signed health including integrity state.	The only evidence-signing key, on the collector host. Wrapped at rest (argon2 + XChaCha20-Poly1305), or held in a PKCS#11 module where the private key is generated inside the module and never leaves it.	Cannot authorise. Cannot mint coverage it did not have; a key holder can relabel state but not manufacture the coverage a claim needs.
Reconciler	Signed verdict with reason code and evidence grade, committable to an append-only log.	Its own key; a fourth authority.	Cannot observe or authorise. Cannot raise evidence strength.
Tool gateway	Signed *ToolReceipt*: what request the institution's API or MCP gateway actually received, as a commitment over the arguments, under which decision, and what the gateway did with it.	The gateway's own key, operated by the institution.	Cannot be the agent: a gateway anchored under the agent's key is refused.
External witness	Countersigned receipt over a NEXORA log head, from a system the log operator does not control.	Its own key, held by the regulated entity or a third party.	Cannot alter the log. An operator who rewrites history can still sign the new head, but cannot produce a witness receipt for the head they discarded.
Approval surface, epoch issuer, interlock board	Signed record of what a human was shown; signed capability epoch the enforcer admits; signed heartbeat the interlock requires.	Each its own key.	None can produce a verdict, raise evidence strength, or make containment reachable. Each is compared against, never merged into, the core.

2a. One core, five regulated domains — and what each is forbidden to say

An assessor is entitled to ask whether this is a principled accountability model or a set of special cases fitted to a BFSI checklist. The question is fair and it is answerable from the repository rather than from assurance, so we answer it here.

The same core carries **five deployment profiles across separate regulated domains**. None of them has its own reconciler, its own verifier, its own evidence hierarchy or its own trust score; each occupies an ordinary profile id beside the others. And the primary contribution of every one of them is a statement of **what it cannot establish**:

Profile	Tests	What that domain's profile exists to refuse
Financial / enterprise	99	The deployment class this submission concerns. No second evidence hierarchy; the same verdict vocabulary as every other profile.
Uncrewed aerial systems	126	Its stated central contribution is “precisely how little that core can honestly say about an aircraft”.
Robotics autonomy	131	Accounts for the autonomy runtime’s execution. **Motion, contact, object state and task outcome resolve to CANNOT_PROVE** unless a deployment supplies an independent physical mechanism, which this build does not.
Industrial / critical edge	172	Accounts for which binary ran, on which host-boot, which sockets it held, which configuration it wrote. **A flow, a pressure or a completed shutdown resolves to CANNOT_PROVE.**

Profile	Tests	What that domain's profile exists to refuse
Disconnected / space autonomy	102	Collects evidence with no link and verifies it offline after contact. **Delayed arrival is not non-occurrence**: evidence arriving in a second says exactly as much about event time as evidence arriving in a month. Gaps are sticky and record the *exact* missing sequences, because “three segments missing” cannot be checked off later and “segments 7, 8 and 9 missing” can.

Why this is evidence rather than breadth. A system built only for banking can be fitted to a banking checklist, and nothing in it would reveal the difference. A core that must also tell an aviation regulator how little it can say about an aircraft, and an industrial operator that it cannot evidence a pressure, is a core whose limits are structural rather than negotiated. **The discipline the Department is asked to rely on here is the same discipline, in the same code, that four other regulated domains would test differently.**

Scale, for orientation only: **51 crates, ~163,000 lines of Rust, 2,690 tests** in the accountability workspace. Those are inputs, not outcomes, and section 10 is where this brief says what has actually been demonstrated.

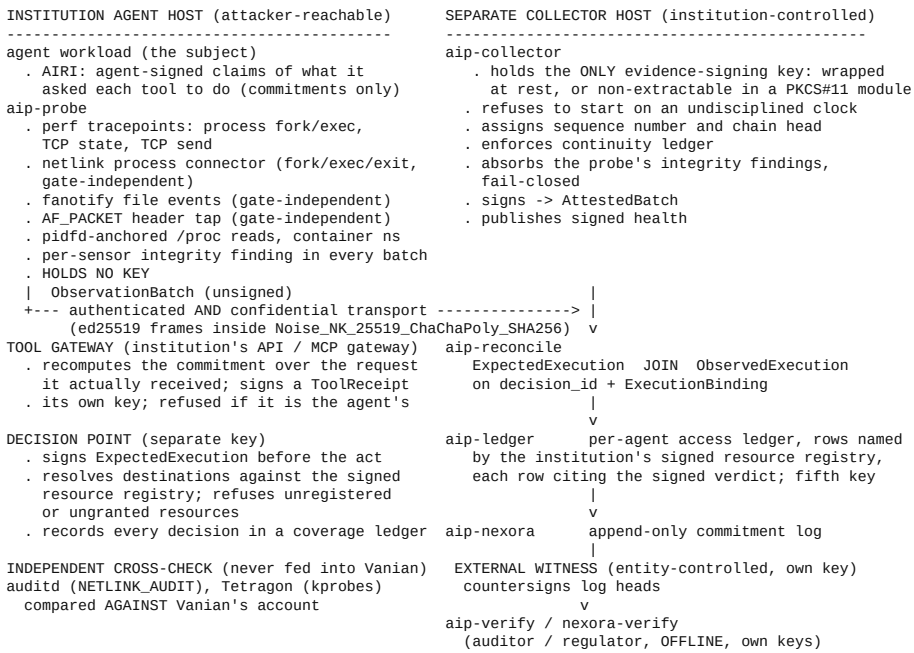
The limit on this section. None of the four non-financial profiles has been deployed either. Section 12 already places the physical-autonomy profiles at TRL 4 and physical autonomous-system accountability at TRL 2–3. Their value to this submission is as evidence that the core’s refusals are principled, **not** as evidence that anything has been run in an aircraft, a plant or a spacecraft.

The mechanisms that make the core distinctive are catalogued for review in `docs/IP/INVENTION_REGISTER.md` — nine entries, each with its problem, mechanism, source location, invariant, tests and the alternative embodiments not taken. That document asserts of itself that **no entry is claimed to be novel, non-obvious or patentable**, that no prior-art search has been performed and no application filed. It is cited here as a map for a technical reviewer, and for no stronger purpose.

3. Deployment topology

The trust boundary is the deployment. Two boundaries carry the argument and must be enforced in the institution's environment, not assumed:

- **Key custody boundary.** The evidence-signing key is generated on the collector host and never leaves it; where a hardware module is used it is generated inside the module, marked non-extractable, and the host only ever sends 32 bytes and receives a signature. The probe on the observed host has no key and no signing method. The gateway key lives at the gateway, the registry key with the institution's infrastructure owner, the witness key on a system the log operator does not control.
- **Evidence boundary.** Independent open-source observers (auditd, Tetragon) are compared against VANIAN's account and never consumed by it. An instrument the system consumed would become part of the system and the comparison would stop being a comparison.



4. What is observed, and what is deliberately not

Observation is from the kernel, out of band of the agent process, using Linux perf tracepoints, the process-event netlink connector, the fanotify queue and an AF_PACKET header tap, with process identity anchored by pidfd rather than by reusable

PID. **Since Revision 2 the pilot probe drives the file and wire sensors too**; they are no longer demonstrator-only, and the last column now says what each costs in privilege.

Event	Evidence recorded	Not recorded	In the pilot probe
Process fork / exec / exit	Kernel-pushed lifecycle event with kernel time; executable content hash and start time under a pidfd anchor; parent lineage; the container namespaces the observer read for itself	Command line, environment	Yes (perf tracepoints and netlink connector)
TCP connection / state change	Peer address and port, state transition, kernel time; the ledger names the peer as the institution's registry names it	Payload	Yes
TCP send	Kernel byte count per send	Message bodies	Yes
Tool invocation (application layer)	Gateway-countersigned receipt: tool identifier, commitment over the arguments, decision cited, disposition	The arguments themselves: amount, beneficiary, account	Not a probe sensor: the institution's gateway signs it
File write	Completed write-and-close on a marked mount, bound to the acting process through a pidfd the kernel created from the event's own process reference; path, device and inode	File contents. Nothing is read through the descriptor	**Yes** (fanotify; needs CAP_SYS_ADMIN, so it is enabled per deployment)
Wire frame	IP and TCP headers, flags, sequence numbers and payload length	Payload bytes	**Yes** (AF_PACKET; needs CAP_NET_RAW)

Content minimisation is structural, not a configuration option: no payload is captured or hashed anywhere in the observation pipeline, and transaction arguments enter the mandate, the receipt and the agent's claim only as domain-separated commitments. Repository tests serialise every signed document of a transfer and assert that the amount, the beneficiary and the account appear in none of them; the two sensors added since Revision 2 are held to the same rule by live tests that write a known secret to a watched file and send a known payload over a watched socket, then assert that neither appears anywhere in the resulting batch. Every field that leaves the host is enumerated in a field inventory, and a test fails the build when a new field is added without one. This is the property that lets the pilot proceed under the Digital Personal Data Protection Act with a field-level review by the institution's Data Protection Officer rather than a data-flow redesign. Everything runs on-premises and offline; no foreign cloud sits in the trust path.

5. Authority over named resources, and the access ledger

A capability scope says which actions an agent may take. Inside a regulated entity the question that matters as much is which systems it may take them against: the loan ledger, never the payments switch. Three additions answer it without changing any signed wire format.

- **Resource registry.** The institution signs a versioned registry of its named systems: address ranges, ports, protocol, and a criticality class (informational, operational, transaction-critical). Overlapping entries are refused at signing. A verifier resolves any endpoint to exactly one name or to the explicit outcome *unregistered*.
- **Resource grants in the scope.** A grant to a named resource is an ordinary entry in the agent's signed capability scope, so it attenuates down a delegation chain exactly as an action does: a sub-agent cannot hold a resource its parent did not. The decision point resolves the request's own signed destination against the registry and refuses *DestinationUnregistered* and *ResourceNotGranted*. An institution that configures no registry gets no resource check, and the decision point says so; the pilot configures one.
- **Per-agent access ledger.** For an agent and a closed window, a signed ledger of every endpoint it was observed reaching, from attested observer batches only, each row named by the registry revision it cites and carrying the reconciler's signed verdicts for that row. A row's standing (verified, adverse, unproven, no verdict) is a fixed rule over the cited verdicts; a verifier recomputes it and refuses a ledger whose rows were renamed, dropped, reordered or restated. The ledger is committed to the append-only log as its own leaf kind and verifies offline. It is the evidence attachment for the AI inventory the FREE-AI framework asks the institution to keep, and it is what an inspector reads instead of an address list.

6. The tool boundary: authorised, received, claimed

After a disputed transaction a supervisor asks one question the kernel cannot answer: was the request that reached the payments system the request that was authorised? The kernel observer sees a socket to the gateway and never the application payload. The party that can answer is the institution's own tool gateway, which holds the request bytes and is not operated by the agent.

Document	Signer	Says
ExpectedExecution (tool-invoke class)	Decision point	What was authorised: a tool identifier and a commitment to the canonical arguments, sealed into the parameter digest.
AIRI ToolInvocation	The agent	What the agent says it asked for. A claim, never evidence.
ToolReceipt	Tool gateway	What the gateway actually received, with the commitment recomputed by the gateway from the request bytes; the decision cited; the disposition (forwarded, refused, failed).

The comparison recomputes the sealed digest from the verified receipt through the same function the decision point used, so every evaluated answer is arithmetic over two signed documents that a third party recomputes offline. The named outcomes:

- **Receipt does not match authorisation.** What reached the system is not what was authorised, whatever the agent claimed. This is the disputed-transaction class, and the agent's account is not what the finding rests on.
- **Claim does not match authorisation.** The gateway received what was authorised; the agent misreported its own request.
- **Agrees**, recording whether the agent's account agreed too, or **not evaluated** with a reason. A decision with no receipt is a coverage statement, never a finding: most actions are not tool invocations, and a gateway that emits no receipts is a gap to be stated, not scored.

Two limits, stated. A receipt says what reached the gateway, not that the downstream effect occurred; the switch may have acknowledged and failed. And the finding is surfaced beside the reconciler's verdict rather than inside it: wiring a tool-boundary divergence into the verdict path is a gated decision with the same four conditions the approval tier records, and it has not been taken.

7. What the reconciler can say

The verdict vocabulary is closed. Each verdict carries a machine-readable reason code and an evidence-strength grade that is strictly separated from severity: a serious-looking finding on weak evidence stays weak.

Verdict	Meaning	Requires
VERIFIED_EXECUTION	Authorised action observed executing as authorised, by the bound process.	Signed expectation and signed observation joined on decision identifier and execution binding.
UNATTRIBUTED_EFFECT	An effect was observed for which no authorisation exists.	A signed decision-point coverage statement proving the decision list for the window is complete. Without it the system cannot distinguish 'no authorisation' from 'authorisation not seen'.
PHANTOM_CLAIM	An authorised action was claimed but never observed executing.	Attested observer coverage of the window; otherwise it degrades to CANNOT_PROVE.
IDENTITY_DIVERGENCE	Execution occurred, but by a process other than the one bound.	Execution binding on both sides. Where both sides recorded container placement and it differs, that is a positive finding; where only one side could read it, the answer is 'not comparable', never a divergence.
CANNOT_PROVE	The default. Observer absent, degraded, blinded, or the window not covered; or an action class the kernel cannot evidence, such as a tool invocation's arguments.	Nothing. This is what the system says when it does not know.

Two consequences follow. First, a blinded or interfered-with observer produces a signed integrity failure and the reconciler answers CANNOT_PROVE with an explicit reason; it never manufactures a PHANTOM_CLAIM against an innocent workload. Second, the decision point's coverage statement is computed from its own ledger and never supplied by a caller, and once issued, the decision point refuses to issue a later decision into that closed window. A completeness claim, once signed, cannot be quietly falsified afterwards.

8. Enforcement, interlock and approval tiers

Three further components exist in the repository. They are kept out of the observation and reconciliation path by construction, and a test in each reads its own manifest to keep it so. None produces a verdict or makes autonomous containment reachable.

Tier	What it does	Fail-closed rule	Status
Enforcement (execve gate)	A supervisor in the workload's syscall path, via seccomp user-notify, permits a program to execute only while a signed capability epoch naming its content hash is in force. The epoch carries a strictly increasing counter.	No epoch, an expired or replayed epoch, an unhashable program, or a dead supervisor: deny. If the supervisor dies the kernel fails the pending syscalls.	Built with adversarial tests. The live seccomp test needs a capable host; not yet run on institution hardware. Bypassed by an attacker who already has root.
Interlock (dead-man's switch)	Inverts the burden: the host must continuously prove, with a signed counted heartbeat, that it is in a condition where actuation is permitted. A signed 'I am blind' from the observer trips it faster than silence.	Tripped is absorbing; only an operator resets. Arming is refused if worst-case trip time exceeds the actuator's declared stopping budget.	Protocol and state machine built and tested in software. The hardware controller and its monotonic counter are specified, not deployed.
Approval (shown versus authorised)	Compares a signed record of what a human was shown and approved against the signed expectation the decision point sealed. Catches a genuine approval of something other than what was authorised.	No approval record is 'not evaluated', never a finding; a record for a different decision is not compared.	Built with adversarial tests. Surfaced beside the verdict; engine integration gated.

Relevance to the draft Model Risk Management guidance: the kill switch it asks for is the combination of a revoked capability epoch and a withheld heartbeat. Both exist as fail-closed mechanisms. The trigger from a verdict to a revocation is the institution's orchestration or SOAR, by design; in the pilot the system stays advisory.

9. Security model

- Ed25519 signatures (RFC 8032) and SHA-256 over canonical, length-prefixed encodings with domain separation, so a signature over one artefact type cannot be presented as another. Every field a verifier reads is inside the signature that vouches for it.
- Observation batches are sequence-numbered and hash-chained at the collector; a rolled-back or replayed batch breaks the chain and the collector refuses to sign, leaving a signed report.
- **The probe-to-collector link is confidential as well as authenticated** (new since Revision 2). The authenticated protocol runs inside a Noise_NK_25519_ChaChaPoly_SHA256 session in which the probe pins the collector's static public key, so a capture of the link reveals neither the probe's identity nor any observation metadata. The acceptance test is a packet capture asserted to contain none of it, with a plaintext control proving the search would have found it.
- **Signing keys may live in a hardware module** (new since Revision 2). A PKCS#11 custodian generates the key inside the module as sensitive and non-extractable and reads the policy back from the module rather than assuming it; the process sends 32 bytes and receives a signature. There is no import path, because a key that was once outside the module cannot be shown never to have been copied.
- Fail closed for the verdict, fail open for availability: missing or ambiguous evidence yields CANNOT_PROVE; an observer fault never disturbs the institution's workload.
- Offline verification: the verifier is a pure function of the artefact and the institution's trusted keys. There is no call-home in the trust path. **Each of the three evidence families now has one** (the third is new since Revision 4): `aip-verify` for evidence packets, `nexora-verify` for append-only log heads and proofs, and `aip-observe-verify` for the observer's own chain of attested batches and signed health records. Release artefacts are signed, not only checksummed.
- Independence checks a verifier can make when it knows the agent's key: a decision point, an observer, a gateway or an approval surface anchored under the agent's own key is refused.
- **Log heads are countersigned by an external witness** (new since Revision 2). An append-only log proves inclusion and consistency, but not that today's root is the root that existed yesterday: whoever holds the log key can build a shorter log and sign it, and every internal check still passes. A witness on a system the log operator does not control, holding a key the operator does not have, countersigns heads. The operator can still sign a rewritten head; they cannot produce a witness receipt for the head they discarded.
- **The collector refuses to start on an undisciplined clock** (new since Revision 2). It stamps and signs every attested batch, so an unsynchronised clock there does not produce one wrong timestamp but an archive that is wrong together and internally consistent, which nothing in the sealed bytes afterwards reveals. CERT-In Direction 20(3)/2022 requires NTP-synchronised clocks; a reference chrony configuration pointing at NIC and NPL servers ships with the pilot kit.
- Service layer: bearer keys stored only as hashes, bound to tenant and role; tenant taken from the key, never the request; append-only evidence store with no delete path. Revocation is tenant-scoped and consulted by evidence verification. Request body size, per-request timeout and a global in-flight ceiling are enforced in process, with TLS and per-client rate limiting at the institution's proxy and a reference configuration supplied.

- Storage controls are enforced by the database under a role the service does not hold: the service's login has SELECT and INSERT only, triggers make evidence append-only and enforce legal hold against the owner too, row-level security scopes an unfiltered query, and a 180-day retention floor is a constraint rather than a policy. A restore drill dumps a hardened vault, restores it into a separate empty cluster and asserts that every one of those controls survived the restore.

9a. Cryptographic agility and the evidence horizon

This submission asks the Department to treat signed artefacts as evidence that can be re-verified later. Section 13's storage controls put a **180-day CERT-In retention floor** in the database as a constraint, and a legal hold that outlives retention. An artefact held for a fraud proceeding may need to verify years after it was signed. So the question *how long does a signature on this evidence remain meaningful* is not academic here, and we would rather answer it than be asked it.

Every signature in this system today is Ed25519 over SHA-256. There is one implemented algorithm. That is a deliberate choice — one construction, one verifier, no negotiation — and it carries a stated exposure: **the non-repudiation of retained evidence has a horizon tied to Ed25519.** Unlike confidentiality, this cannot be repaired after the fact. Evidence already signed and retained cannot be re-signed into safety by a future migration; only evidence signed after a migration benefits from it.

What is already correct, and why it is the hard part

Migration fails in practice not for want of a new algorithm but because the old format cannot express which one was used, or because a verifier quietly treats an unknown algorithm as the one it knows. Neither is true here.

- **The algorithm identifier is inside the signed bytes.** Every signature carries one of three values — Ed25519, ML-DSA-65, hybrid — and it is covered by the signature, so it cannot be rewritten after the fact. The wire format does not need to change to carry a post-quantum signature; only the implementation does.
- **An unimplemented algorithm is refused, never coerced.** A signature labelled ML-DSA-65 or hybrid does not get checked as Ed25519 and does not come back 'invalid'. The verifier returns an **error** — it declines to answer about bytes it was never asked to read. The rule is that a label may never become a capability, and it is a test, not a convention: `aip-nexora/tests/audit.rs` a07, with a companion proving the same refusal reaches the package level so no component can smuggle an unimplemented algorithm past the verifier.

That combination is what makes a future migration a change of implementation rather than a change of evidence format, and it is why a verifier from either era cannot silently accept the other era's artefact.

The hybrid path is now built and exercised

Since Revision 9 the migration is no longer only specified. **`aip_crypto::hybrid` signs under Ed25519 and ML-DSA-65 together** (FIPS 204, via RustCrypto's `ml-dsa`, pinned exactly), and **verification requires both halves**. Accepting either would defeat the construction: a break in Ed25519 alone would then forge a hybrid signature and the second algorithm would be decoration. 15 tests, and the properties that matter are the ones that could go wrong:

- **Neither era can be mistaken for the other.** Because the algorithm identifier is inside the signed bytes, the Ed25519 half of a hybrid signature does not verify as a standalone Ed25519 signature, and a standalone signature relabelled `Hybrid` and padded with ML-DSA-shaped bytes is caught — **by the mature algorithm**, not only by the new one.
- **Nothing already sealed changes.** The Ed25519 signing and verification paths are untouched; the hybrid path is additive. Every artefact sealed before this existed verifies byte-identically, which is the whole point — a migration that invalidated the history it was meant to protect would be worse than none.
- **The policy belongs to the verifier, not the artefact.** An institution runs *transitional* (accept either) or *hybrid-required*. A strict verifier refuses an Ed25519 artefact as **insufficient, not invalid** — a correctly signed document under a superseded algorithm must not be logged as a bad signature, or an estate mid-migration reads as though it were under attack.
- **Demonstrated end to end on a real signed document:** an institution's route map (section 6) sealed hybrid, serialised, reloaded, and verified offline under a strict policy; the same map sealed Ed25519 still accepted under a transitional one; a route edited after hybrid signing still caught.

What has NOT been done, stated plainly

- **The four evidence-signing authorities are not migrated.** The collector, the decision point, the reconciler and the gateway all still sign Ed25519. The primitive and the transition policy exist and one document type is exercised end to end; rolling that through the evidence families, the NEXORA log and the three offline verifiers is mechanical but **unbuilt**, and until it is built no evidence artefact carries a hybrid signature.
- **Standalone ML-DSA-65 is still refused.** It remains a reserved identifier. A post-quantum-only signature would trade a mature implementation for an unaudited one rather than adding to it.

- **The ML-DSA implementation is pre-1.0 and not independently audited.** `ml-dsa` 0.1.1 is pinned exactly for that reason, and the hybrid construction is the mitigation: a defect in that implementation cannot on its own produce a signature that verifies, because the mature half must verify too.
- **The post-quantum work elsewhere in our codebase does not close this.** Our network layer uses Kyber-768 for key establishment. That is a **key-encapsulation mechanism — confidentiality of a session** — and it gives no post-quantum signature path. Conflating the two would be the kind of claim this brief exists not to make.
- **Key custody for a second algorithm is unaddressed.** The PKCS#11 custodian of section 13's B4 was exercised against an Ed25519 key; ML-DSA support in a given HSM is the module vendor's, and the institution's choice of module would constrain the migration. The ML-DSA key is derived from a 32-byte seed, so the *shape* of custody is unchanged, but that is not the same as having exercised it.
- **No migration has been scheduled or costed**, and no performance cost has been measured. An ML-DSA-65 signature is 3,309 bytes against Ed25519's 64, which is a storage and bandwidth change an institution should size before committing.

What we propose. The decision belongs to the institution, not to us, because it is their retention obligation and their evidence. We ask only that the pilot's evidence-handling agreement records the horizon explicitly, so nobody later discovers it in a proceeding. Where an institution's obligation is long enough to need it, the migration is a dual-signature epoch — artefacts signed under both algorithms for a transition window, with verifiers required to accept either and the log recording which — and that is specified work, not built work.

10. What has been demonstrated, and what has not

Every claim below corresponds to an executed attack with recorded output, or to an automated adversarial test. The adversarial ledger holds one row per attack with the exact command and the classification the stack actually emitted; **since Revision 2 a single command reproduces every row of it that the host's privileges allow**, reports the named reason for every row it cannot run, and fails if a detection the ledger claims no longer holds. Continuous integration runs it.

What a demonstration here is worth, said plainly. Three limits apply to everything in this section and none of them is removed by adding more tests. First, every attack in the ledger is an attack *we* designed; a suite whose author picks both the property and the adversary establishes that those properties hold against those adversaries, and nothing about completeness — which is why an independent CERT-In empanelled assessment is on the blocker list and stays there. Second, most of the suite constructs its inputs in one process; the smaller set that runs against a real kernel, real separate processes and real system calls is marked as such. Third, and most important: **nothing in this repository has run on a regulated entity's hardware, and no agent performing an institution's real work has been observed.** Since Revision 5 the framework on the other end has been real and not ours -- an unmodified LangGraph 1.2.11 graph -- but the task it carried out was written in order to be observed, and the load was not representative. The strongest rung reached anywhere is a live demonstration on a development virtual machine. A four-rung scoring of every property this system asserts, class by class against the failure modes in the risk map, is published in the repository as WHAT_THE_TESTS_PROVE.md, and it reaches the same conclusion this paragraph does.

Maturity: the software accountability core is at TRL 5 — validated in a relevant environment — and now at the 5/6 boundary. We state it differentiated rather than as a single number, because a single number for this programme would be wrong in one direction or the other. The core that this brief describes — the observer, the off-host collector, the reconciler and the evidence chain — has been validated against a real Linux kernel, with an independently built external observer, under adversarial scenarios including a reproduced attack on the observation path itself and a live attack against the two real daemons; and, since Revision 5, against an unmodified third-party agent framework and an external attack corpus. A real Linux host running real workloads is a relevant environment for *software* autonomy, which is what TRL 5 requires. The same assessment puts our physical-autonomy profiles (UAS, robotics) at TRL 4, and physical autonomous-system accountability at TRL 2–3 until a machine is actually in the loop; none of that is in scope for this brief and we do not carry its readiness into this one.

We are not claiming TRL 6, and the two remaining conditions are specific. TRL 6 requires a fully functional prototype demonstrated in a relevant environment and *partially integrated with existing systems*. The prototype exists, the environment is relevant, and integration with existing systems is no longer absent: an unmodified third-party agent framework, the MCP tool protocol, a PKCS#11 module, PostgreSQL and the Linux audit subsystem are all on the integrated side of that line. Two conditions are not met. **(i) A non-synthetic agent under representative load.** The framework on the other end is now real; the work it performs is not, and the load is not representative. That is blocker B8 expressed in readiness terms rather than deployment terms, and it is precisely what the pilot in section 12 exists to supply. **(ii) Independent corroboration of our observations by a runtime observer we did not build. This condition has been met since Revision 5** and is the substantive change in this revision: two independently built observers -- one of them reaching the kernel by a different mechanism than ours -- watched the same kernel over the same window, and neither recorded an execution absent from AIP's signed evidence. Subsection 10a gives the numbers, the two defects the run found in our own comparison harness, and what

remains open on that front. **Condition (i) therefore stands alone, and it is the one only a pilot can supply.** Until it holds the honest number is 5. We would rather be asked to defend a 5 we can evidence than a 6 we cannot.

- **Foreign eBPF tracepoint gate** attached to each perf sensor (an attacker with CAP_BPF): the observer signs an integrity-unknown state and the reconciler answers CANNOT_PROVE with reason ObserverIntegrityUnknown. Control run with no attack yields no integrity claim and no false positive.
- **That attack has now been executed against the two real daemons** (new since Revision 3; Revision 3 listed it as outstanding). `aip-probe` ran with four kernel sensors open and no key, shipping over the authenticated and confidential transport to `aip-collector` in a separate process holding the only signing key. With the foreign program attached to `sched:sched_process_exec`, the collector signed **IntegrityUnknown** and withdrew attested coverage; with it detached, coverage returned. The observer's sequence advanced throughout, so this was neither loss nor a continuity break but a live observer stating that it could not vouch for its own capability. A single attach suffices: the BPF array is keyed on the tracepoint, so one attach blinds every perf consumer of it host-wide. Rows OI-13 to OI-15 of the adversarial ledger, with the signed health archive as raw output, reproducible by one script. **It ran on a development virtual machine; the same run on a regulated entity's own hosts is still what Phase 4 asks for.**
- **The signed evidence survives the acknowledgement, and an outside party can check it** (new since Revision 4). This is stated as a demonstration rather than a feature because the defect it fixes was invisible to testing: the collector attested each batch, acknowledged it by chain position, and discarded it, so every library test of offline verification passed while nothing was left on disk to verify. Every attested batch is now appended to `attested.archive.jsonl`; the collector publishes its evidence public key as `observer.pub` beside its state, which previously could be obtained only by unwrapping the private key with the passphrase -- exactly what key custody exists to prevent. `aip-observe-verify`, given those two files and that public key and opening no socket, checks each signature and then the chain that links them: positions contiguous, and each record's predecessor hash equal to the **recomputed** hash of the record before it, so there is no supplied digest to trust. It also compares each signed health record's head against the attested batch at the same position -- both signatures are individually valid, and only that comparison would expose a collector that signed two different answers about one moment. Ten ledger rows (EA-01 to EA-10) cover wrong key, edited observation, a record deleted from the middle, reordering, duplication, a corrupt line, an empty archive and a disagreeing health head; deletion and reordering leave every surviving signature genuine, so only the chain check catches them. The live run ends by verifying the archives the running collector just wrote and then refusing the same archive with one record removed.
- **The collector's clock discipline is now inside the signed bytes** (new since Revision 4; Revision 4 listed it under 'not demonstrated'). The collector stamps and signs every batch and every health record, so the clock behind those stamps is the accuracy of the whole archive. It was checked at start and on every health cycle and the answer went to a log — and a log is not evidence anyone holds. The kernel's own view (whether a time daemon was steering the clock, and the kernel's bound on the error) is now part of what the key signs. The specific failure this addresses: an undisciplined collector does not produce one wrong timestamp, it produces an archive that is wrong together and perfectly self-consistent, which nothing in the sealed bytes would otherwise reveal. Editing, deleting or inserting the reading after signing each fails verification; a record that carries no reading is encoded distinctly from one that carries a good reading, so a collector that never looked at its clock cannot pass as one that looked and found it fine. Nine ledger rows. **What it does not buy:** root on the collector host can clear the kernel's synchronisation flag on a clock it has just moved. This raises the bar from trusting an operator's log to trusting a signature, and no further.
- **The signing and verification path has been measured** (new since Revision 4; Revision 4 declined to quote any figure). On a four-core Intel Xeon at 2.80 GHz, single-threaded, with a 139-byte batch: attesting one batch takes **395 microseconds** (about 2,500 per second) and verifying one offline takes **46 microseconds** (about 21,700 per second), so verifying a retained archive of one million attested batches takes roughly **46 seconds** on one core, and verification parallelises trivially. The figure worth the Department's attention is the decomposition: of those 395 microseconds, 43 is signing and chaining and **351 — 89 per cent — is writing the new chain position to disk**. Attestation is bound by storage, not by cryptography, so a deployment that needs more throughput changes the storage path and not the signature algorithm. Raw output with its host line is sealed in the repository and reproduced by one command. These are in-process figures that exclude the network and the probe's own sampling cost; they are a floor for planning and not a service level.
- Forged observer identity, altered observation after signing, altered measured boot after signing, unanchored key version, and deserialisation of a record from nothing: each is refused.
- Signed reconciliation verdict: mutation of result or reason, swap of reconciler identity, and domain confusion are each detected; the verdict can be committed to an append-only log and verified offline.
- Decision-point coverage statements from a ledger, with the closed-window refusal described in section 7, making UNATTRIBUTED_EFFECT reachable on the live path.
- Resource authority: a signed registry with overlap refusal; unregistered and ungranted destinations refused at the decision point; a delegation hop that tries to widen a resource grant refused; an unanchored, impostor or tampered registry refused.

- Access ledger over the real engine: a verified connection becomes a named row citing the signed verdict; renaming, dropping, reordering, re-pointing the registry revision and restating a row's standing are each refused; the ledger commits and verifies offline as its own leaf kind.
- Tool boundary: a gateway receipt for other parameters or another tool is the named divergence regardless of the agent's claim; an agent misreport over a correct receipt is named separately; no receipt is never a finding; unanchored, agent-keyed, tampered and impostor receipts are refused; the arguments appear in no signed document.
- **Container placement cannot be manufactured from blindness** (new since Revision 2). Two observers that both read the namespaces and disagree is a positive finding; exactly one having read them is 'not comparable', never a difference; and the placement is deliberately excluded from the identity digest so a privileged observer and an unprivileged one produce the same digest for the same process. Eight ledger rows cover this class.
- **Verdicts and access ledgers are stored and served** (new since Revision 2; Revision 2 listed this as not demonstrated). The evidence API accepts a signed verdict or ledger only after verifying it under the caller's anchors, returns it verbatim so the answer never depends on trusting the service, and scopes both by tenant.
- Enforcement, interlock and approval tiers: each with an adversarial suite (replayed epoch, counter rollback, forged heartbeat, withheld condition, approval for another decision, stale artefact) and a manifest test that keeps it out of the observation path.

Not demonstrated. Stated plainly so the evaluation can target it:

- **Root or kernel-level compromise of the observed host.** A root attacker can hide a tracepoint gate from the detection query, detach the seccomp supervisor, and clear the kernel's clock-synchronisation flag on a clock it has just moved. The off-host collector bounds tampering after collection; it does not make the host kernel trustworthy. Remedy paths are hardware-rooted or fully off-host observation, not yet built.
- **The overhead this system imposes on an institution's own workload.** The signing and verification path has been measured and is quoted in section 10, and the evidence store has been measured separately. What has not been measured, and cannot be measured honestly from a synthetic benchmark, is what the observer costs the agent it is watching. That is a pilot measurement on your hosts against your agent, and it is one of the things a pilot is for.
- **Autonomous enforcement from a verdict.** Containment is designed but stays off. The enforcement and interlock tiers act on epochs and heartbeats, not on verdicts; no production evidence path today yields the evidence grade that verdict-driven containment would require. The system is advisory.
- **Hardware attestation, TPM/TEE, hardware monotonic counter, post-quantum cryptography.** Not deployed.
- **Operation in a transaction's critical path.** The observer never gates or blocks a transaction and is not designed to. The execve gate, where an institution chooses to run it, gates program launch, not transactions.
- **Kernel-dependent tests in continuous integration.** CI runs unprivileged and the kernel sensor and seccomp tests skip there. Evidence for them is developer runs on capable hosts and the recorded attack ledger, not a green CI badge.
- **Gateway integration at the institution.** Since Revision 5 a working reference MCP gateway ships and signs receipts, with an adapter that calls it rather than reimplementing the cryptography. What remains is extending the *institution's own* API or MCP gateway to do the same. That is still integration work at the institution; what has changed is that there is now a reference implementation to integrate against and to test a candidate gateway's receipts for equality with.

10a. Independent corroboration by observers we did not build — executed

There is one criticism of everything above that we could not answer with more of our own tests, and we would rather state it than be asked it: **every sensor that had ever agreed with this system was a sensor we built.** The cross-sensor divergence check of section 9 is real and load-bearing, but it compares two of *our* sensors against each other. An assessor is entitled to ask whether a runtime observer built by someone else, watching the same kernel at the same moment, records what we say it records. **That has now been run.**

Two independently built observers ran alongside AIP's probe and collector over the same kernel and the same window. **bpftrace 0.20.2** is an independent eBPF implementation attached to the same `sched:sched_process_exec` tracepoint AIP's perf sensor reads. **auditd 3.1.2** reaches the kernel through the audit subsystem — a different mechanism entirely. Both results are sealed in `validation/external/corroborate/results/sealed/` and reproduced by one script.

Independent observer	Independence	Agreed	AIP only	Recorded by them, absent from AIP
bpftrace 0.20.2	independent implementation, **same** kernel mechanism	113	1	**0**
auditd 3.1.2	**different** kernel subsystem	112	2	**0**

Neither observer recorded an execution that is absent from AIP's signed evidence. The "AIP only" column is not a defect: the third-party tracers start and stop as processes of their own and are configured for a narrower scope. The archive was verified under its anchor *before* anything was compared; a corroboration report over unverified bytes is a statement about a file, not a system.

The two kinds of independence are not equal, and the tool does not let them be read as one claim. bpftrace corroborates that AIP faithfully reported what the tracepoint emitted; a foreign eBPF gate on that tracepoint would blind both observers identically, so it does not corroborate coverage. auditd does not share that mechanism and a tracepoint gate does not blind it, which is why it is carried alongside rather than instead. `aip-corroborate` labels every report with which of the two it is.

What the run found — in our own harness

Both defects below were in the comparison, not in AIP, and both are recorded because the way they were found is the reason to trust the result.

- **275 fabricated AIP blind spots, from comparing different things.** auditd audits the execve *syscall* and records every *attempt*: `/usr/bin/env true` makes `env` search `$PATH`, and each miss is an execve returning ENOENT. AIP's scheduler tracepoint fires only on a **successful** exec, which is correct — a failed execve is not an execution and nothing ran. The first run counted the failures against AIP. The split was exact: 106 successes against 106 agreed, and 275 failures with exit=-2. The two classes are now separated, and the attempts are still counted and reported, because "auditd sees attempts, AIP sees executions" is a real difference a reader should know.
- **One apparent gap that was a three-millisecond boundary artefact.** The second run reported a single missing execution: a `/usr/bin/sleep` auditd recorded 3 ms before the window closed. **bpftrace missed it too**, which is what identified it — it was the harness's own teardown, and AIP batches on an interval, so an execution received milliseconds before the probe stops has not necessarily reached an attested batch. The window now closes at the end of the workload, with both observers still running through a settle period.

Neither correction weakens a verdict, and each is guarded by a test: the first stops the tool demanding that AIP record executions that never happened, the second stops it measuring a flush race instead of observer agreement, and a further test proves the exclusion cannot hide a genuinely missed *successful* execution.

What is still open on this front

Falco and Tetragon remain the first-choice observers and were not used. Their distribution channels are unreachable from the environment this was built in — the egress proxy denies them and neither is in the distribution archive. The comparison core is source-agnostic and already carries a Falco variant; an adapter for its JSON output is the only missing piece. This is stated rather than quietly substituted. Of the remaining items on the corroboration plan, the SPIFFE/SPIRE cross-check **has since been executed** and is subsection 10b. One is still not done: anchoring the evidence chain's head in a **public transparency log** (Sigstore Rekor), so that the chain's existence at a time is checkable by a party with no relationship to us or to the entity.

What this establishes, and what it does not. It establishes that AIP's observations survive comparison with observers it did not author, including one that does not share its kernel mechanism. **It does not make the host kernel trustworthy.** Two differently-built observers raise the cost of a consistent lie; they do not eliminate it, and where both read the same mechanism one gate blinds both. It is also a single run of ~113 executions over a few seconds on a development virtual machine: not a soak test, not representative load, and not an institution's hardware. That limit is the same one stated in sections 9 and 11 and is not narrowed here.

10b. Identity corroboration: cross-checked against SPIFFE/SPIRE

Subsection 10a corroborates what AIP *observed*. This corroborates what AIP *binds* -- its claim that a given process instance is running given code -- against **SPIFFE/SPIRE**, the CNCF-graduated workload-identity standard, built from source at v1.15.3 and run standalone. The two systems answer adjacent questions: SPIRE answers *which named workload is this process*, AIP answers *what code is actually running in this process instance*. Neither asks the agent.

The join is the executable's SHA-256. SPIRE's `unix` workload attestor, configured with `discover_workload_path`, offers it as a `unix:sha256` selector; AIP measures the same quantity independently from the kernel. **SPIRE issues an SVID for such an entry only if it measured that digest itself**, so the SVID's existence is SPIRE's own assertion about the workload's code.

Case	Registry compared against	Result
A live workload, both authorities watching	the approved registration	**AGREES** -- same digest, same process instance
The identity re-pointed at different code	SPIRE's *current* registration	**AGREES** -- SPIRE remains self-consistent

Case	Registry compared against	Result
The identity re-pointed at different code	the **approved** registration	**DIVERGES** -- DigestMismatch

The third row is the finding: a workload identity registration was changed to point at code that was never approved, and the running system no longer matches the change-controlled record. **Nothing is fabricated to produce it.** Both SVIDs were genuinely issued by SPIRE and both measurements are genuinely AIP kernel observations; the only difference between the second and third rows is which registry record the comparison is made against, and both are real `spire-server entry show` output captured at different times. The second row exists so the result is falsifiable: a tool that reported divergence in both directions would be worthless.

The soundness control. The registration carried two selectors, the digest and `unix:uid:0`. If uid alone had been doing the matching, every root process would have received the identity and the digest would have been decoration -- and the comparison above would rest on nothing. So a byte-for-byte copy of the same program, one byte changed, run as the same user, was **refused** by SPIRE. The digest is load-bearing. Where an entry rests on no digest at all, the comparison answers *not comparable* and never *agrees*.

Two defects this exercise found -- in our code

- **A control that cried wolf on a healthy workload.** The first joint run reported an ordinary process as a reused pid, because the pid-reuse guard was anchored to the SVID's `notBefore` and the process had started twelve seconds after it. The cause is that **SPIRE pre-mints SVIDs for registered entries before any workload asks**, so `notBefore` records minting rather than attestation; the instant SPIRE attests a particular caller is when that caller fetches. The guard now compares against that, and two tests lock both directions. A control that false-positives on every normal deployment is one that gets switched off in its first week, which is worse than not having it.
- **AIP does not bind every execution, and that is correct.** A run failed because the workload's exec was recorded with no bound measurement: the probe saw the exec but could not complete a pidfd-anchored measurement of that instance, with the probe's own dropped-event counter at zero throughout -- measurement racing, not loss. The comparison answered **not comparable** rather than manufacturing an agreement, which is the designed behaviour. The harness now requires a bound measurement and re-runs the workload otherwise, reporting how many attempts it took; each attempt is an independent genuine execution.

A claim we checked and withdrew

A weakness in SPIRE's workload attestation was reported publicly in September 2026, and we considered demonstrating it beside our own binding. We checked it against its primary source first. It is **post-exploitation research requiring root on the node**, it targets the Kubernetes attestor's cgroup parsing rather than the unix attestor used here, it carries **no CVE and no advisory**, and its authors state it is not a cryptographic or protocol break and matches a trust assumption the SPIFFE project has always documented. We also found that SPIRE's unix attestor reads `/proc/<pid>/exe`, **the same source AIP measures from**, so for that attestor the two share that attack surface and any claim of immunity would have been false. The demonstration was dropped before it was written into this brief. We record it because the Department is entitled to know what we checked and discarded, not only what we kept: both systems document the same limit, which is a hostile host.

10c. The funnel an agent can decline — out-of-band reconciliation

Sections 6 and 10a-b concern what passed *through* the system. There is a class of failure none of them reaches, and it is not closed by making any control stricter: **an agent that does not use the governed path leaves no record to be missing from.** The tool boundary adjudicates calls that arrive at it. A stricter boundary is still one an agent can route around, and inline enforcement cannot see its own bypass. Catching that requires comparing the ledger against a sensor outside the governed channel.

A reconciler for exactly this exists in our codebase and has for some time. Its weakness was its input: it reads a log of process launches and network egress, and in the deployment it was designed for, that log is written by the components being watched. **A tool server's own access log is the tool server's account of itself** — the weakest possible root for a detector whose entire purpose is to catch something that is lying about what it did.

AIP observes process launches and connections **from the kernel**, binds them to process instances through pidfd, and has an off-host collector sign the result. `aip-sentinel-feed` renders that evidence as the reconciler's input, replacing the self-reported root with a kernel-derived and independently signed one. It verifies the archive under the observer's anchor **before rendering anything** and writes nothing at all if that fails; an effect log derived from unverified bytes would be a statement about a file rather than about a machine, and it is handed to a detector that raises critical findings from it.

This is not corroboration and we do not present it as such. The reconciler on the far side is ours. Sections 10a and 10b are corroboration because bpftrace, auditd and SPIRE were built by other people; this is an architectural gap closed with our own components, and the Department should weigh it as that and nothing more.

What was actually checked

The claim that two components interoperate is worth nothing asserted. The feed's output is written to disk and read back by the reconciler's own reader, against the real agent archive of section 10, not a fixture:

Check	Result
Every line of the real feed parses, through the consumer's own reader	**0 malformed** of 22 records
Spawn records agree with the delegation chain derived from the same archive	**21 and 21**
An unapproved destination becomes a finding naming it and citing the record id	Yes; the measured 81,920 bytes survive into the finding
An approved destination produces no finding *(soundness control)*	**None** — the detector is not simply always red

The number in this section the Department should read first

Building the feed's coverage report exposed a defect in the report itself, and fixing it produced the most useful measurement in this revision. The first version counted only events that reached the attribution stage — and unbound events are discarded before it, correctly, because an unbound event is not evidence of a process instance. **Eight connection attempts in the real archive were therefore counted nowhere, while the report implied full coverage.** A report that undercounts what it could not see is worse than no report, because a reader takes its silence for coverage. It now counts them by a direct scan, and two tests assert the accounting balances exactly.

What that then showed, on the same real agent run: **84 of 105 `execve` events could not be bound to a measured process instance.** The workload was a shell spawning short-lived children that exited before a pidfd-anchored measurement could complete. This is the behaviour section 10b already reports — the probe saw the exec and could not measure that instance — and nothing here is new about the mechanism. What is new is its **magnitude on a workload of that shape**, which no previous artefact of ours quantified.

We state it plainly because it is the honest answer to a question an assessor should ask: *what fraction of execution does this observer actually bind?* On short-lived processes, a minority of it. The system's response is to report the remainder as unmeasured rather than to guess — which is the CANNOT_PROVE discipline of section 5 operating exactly as specified, not failing — but a control that binds a fifth of execs on one workload shape and most of them on another is a control whose coverage must be measured per deployment and not assumed. **A pilot must report this ratio for the institution's own workload.** Raising it is probe-side engineering (earlier pidfd acquisition), and it is on the roadmap rather than in this brief as a solved problem.

Reproduce: `cargo test -p aip-sentinel-feed` (34 tests) and `cargo test -p aip-feed-conformance` (4). The sealed feed and its full coverage report are at `integrations/agents/results/sealed/effect_feed.jsonl`.

Demonstrator versus pilot deployment

Two configurations exist and the difference matters for what a pilot can conclude. Since Revision 2 the gap has narrowed to the evidence key's location and the integrity finding's path, which are the two that carry the security argument.

Property	Single-host demonstrator	Two-host pilot daemons
Sensors	Perf tracepoints, netlink connector, fanotify, AF_PACKET	**The same four**, with fanotify and AF_PACKET enabled per deployment for their capability cost
Gate-independent coverage	Yes	Yes: the connector, plus fanotify and the wire tap, none of which a per-tracepoint eBPF program can silently gate
Integrity finding path	Sensor to signer in one process	Inside the signed batch, probe to collector; collector fails closed on silence
Evidence key location	Same host as the sensors	Separate collector host; wrapped at rest or non-extractable in a PKCS#11 module
Transport	In-process	Authenticated and confidential; a capture shows no observation metadata
Verdict classes reachable	All that the evidence supports	Same; PHANTOM_CLAIM needs the connector's process-lifecycle coverage, which the pilot runs
Resource naming and ledger	Library-level; same code	Same code; registry signed by the institution's infrastructure owner before Phase 2

11. Threat model

Attacker	Capability	System stance
A1 confused agent	Unintended actions, no evasion	Baseline. The common real case; detected as IDENTITY_DIVERGENCE, UNATTRIBUTED_EFFECT or an unregistered destination where coverage allows; a mis-sized transfer surfaces at the tool boundary when the gateway signs receipts.
A2 adversarial agent, unprivileged	Evades, misreports, no root	In scope. Expected outcome CANNOT_PROVE or IDENTITY_DIVERGENCE; never a silent VERIFIED_EXECUTION. A misreported tool call over a correct receipt is named as such; a forged receipt fails because the agent does not hold the gateway key.
A2+/A3 agent with CAP_BPF or root	Attaches a tracepoint gate, reads local state, alters the clock, detaches the supervisor	Detection bound. A CAP_BPF gate is detected. Full root defeats on-host detection and on-host enforcement; the off-host collector bounds post-collection tampering only.
A4 compromised observer	Controls the sensor	Key custody and the coverage predicate prevent a forged observation from minting UNATTRIBUTED_EFFECT against an innocent workload. A holder of the collector key can relabel state but cannot mint coverage it never had. An observer that cannot read what it is asked about records that it could not, and the comparison answers 'not comparable' rather than turning blindness into a finding.
A5 compromised gateway or registry signer	Signs false receipts or a false registry	Out of the observer's reach by design: a receipt is the gateway's claim about itself and a registry is the institution's. Both are refused if unanchored or agent-keyed; their custody is the institution's key-management control, and the brief does not claim otherwise.
A6 log operator	Rewrites the commitment log and signs a new head	Bounded by the external witness. The rewritten head verifies internally, but the operator cannot produce a witness countersignature for the head they discarded, and a verifier that requires one detects the truncation.

12. Proposed pilot: narrowly scoped, observer-only, non-gating

A pilot is a falsifiable experiment, not a demonstration. It passes only if the observer's honesty holds under the institution's real conditions, and it fails usefully if it does not.

Scope

- One regulated entity, a small number of Linux hosts running an AI agent that takes consequential actions (loan decisioning, reconciliation, payments operations support are natural subjects).
- **Advisory only.** Autonomous containment off; the execve gate and interlock are not part of the pilot unless the institution asks for them separately. Nothing freezes a process, drops a packet, changes a firewall or touches an actuator. The institution names protected processes exempt from any action.
- **Never in a transaction's critical path.** If the observer degrades, the institution's workload is unaffected; only VANIAN's coverage claim shrinks.

Phases and exit gates (built and validated in dependency order; no phase is skipped)

Phase	Deliverable	Exit gate
1	Off-host collector, host-side probe, authenticated and confidential transport on institution hardware	A real agent process is observed; its batch is signed by a collector on a different host; the verifier accepts it offline. **All three clauses have now been met end to end on a development host, including the third, which required the collector to keep what it signs. What Phase 1 still asks for is this run on the institution's own hardware, with the collector on a genuinely separate host.**
2	Decision point issues signed expectations for the pilot agent's actions against the institution's signed resource registry	Expectation and observation join; the honest path yields VERIFIED_EXECUTION; a connection to an unregistered address is refused before the act; the access ledger for the window names every endpoint reached.
3	Reconciler, ledger and evidence API wired; gateway receipts where the institution's gateway supports them	Verdicts and ledgers stored and retrievable; authority chain rebuildable; tenant isolation holds; a deliberately mis-parameterised tool call yields the named divergence.
4	Adversarial and honesty validation on institution hosts	Blinded observer yields ObserverIntegrityUnknown; observer down yields CANNOT_PROVE; no false PHANTOM_CLAIM; no false divergence with no receipt. **A pilot that skips this has not established honesty, whatever the happy path shows.**

Phase	Deliverable	Exit gate
5	Independent cross-check and auditor handoff	VANIAN's account reconciled against auditd and Tetragon; signed audit-export bundle and access ledgers handed to the institution's auditor; verification reproduced by them with their own keys.

Data handling

No payload captured or hashed. Sends are evidenced by byte count, executions by content hash, connections by endpoint and kernel time, file events by path and inode, wire frames by header and length. Every field that leaves the observed host is enumerated in a published field inventory with its personal-data risk and the reason it is collected, and a test fails the build if a field is added without an entry. Everything remains on the institution's premises.

13. Pre-production blockers, stated up front

A pilot may proceed with these open because it is advisory and non-gating. A regulated production rollout may not. We would rather the FinTech Department see this list now than discover it later. **Seven rows have moved since Revision 2 and are marked; the hard gate, B8, has not.**

#	Blocker	Status
B1	TLS and rate limiting at ingress; confidential probe-to-collector link	**Changed.** The probe-to-collector link is now encrypted (Noise NK) as well as authenticated, and the service enforces body-size, timeout and in-flight limits in process. TLS termination and per-client rate limiting remain the institution's proxy configuration, for which a reference file ships.
B2	Independent security assessment by a CERT-In empanelled auditor, with remediation	Not yet done. Hard gate for every segment. Unchanged.
B3	Durable, backed-up storage with DR/BCP; storage-enforced immutability and retention	**Changed.** Row-level security, delete revocation, append-only triggers, legal hold and a 180-day retention floor are enforced by the database under a role the service does not hold, and a restore drill proves those controls survive a backup and restore. RPO/RTO, off-site custody, backup encryption and drill cadence remain the entity's to set.
B4	KMS/HSM custody for service, collector, ledger and gateway signing keys; key rotation	**Changed.** A PKCS#11 custodian generates a non-extractable key inside the module and verifies the policy by reading it back; tested against a real module through the same interface a hardware device presents. Choice of module and its certification, PIN custody, dual control and key ceremony remain the entity's.
B5	Signed audit-export bundle and standalone verifiers distributed to the auditor	**Changed.** Done, and the earlier caveat is closed: release artefacts are now signed, not only checksummed.
B6	Metrics, monitoring, SIEM export, incident bundle in the CERT-In window, runbook	**Changed.** An authenticated metrics endpoint, CEF-over-syslog export of verified verdicts, an incident evidence bundle and an on-call runbook exist. Connecting them to the institution's SIEM and paging remains integration work.
B7	Control-framework mapping (RBI IT and cyber directions, Outsourcing, Digital Lending, FREE-AI, MRM, CERT-In, DPDP) to code and tests	**Changed.** The conformance program now emits the regulatory control dimension alongside the invariant-by-domain matrix.
B8	Off-host collector and authenticated transport running on institution hardware	**The hard gate, unchanged.** Components are built, including the connector, the file and wire sensors, the confidential transport and the integrity finding across the split. What has not happened is the run on the institution's own hosts, and no amount of test coverage substitutes for it.
B9	Client-side signing so no agent key crosses the wire; agent-framework adapters	Partial, unchanged: agent-signed AIRI events exist as a library; framework adapters not yet done.
B10	Gateway receipt issuance at the institution's API or MCP gateway	Unchanged: format and verification built; gateway-side issuance is institution integration work.
B11	External anchoring of log heads to an institution-controlled second system	**Changed.** An external witness countersigns log heads from a system the log operator does not control, holding a key the operator does not have; a rewritten log cannot produce a receipt for the head it discarded. Where the witness runs, and who holds its key, is the entity's deployment decision.

14. What we ask of the FinTech Department

- **A technical evaluation session** (about 45 minutes) with people who can challenge the system. We will demonstrate the honest path, an interfered-with observer, a refused unregistered destination, a mis-parameterised tool call against a signed receipt, and offline verification performed by your side with keys you hold.
- **Guidance on the appropriate pathway:** the RBI Innovation Hub, the Regulatory Sandbox, or a direct pilot with a regulated entity.
- **If the evaluation warrants it, an introduction to one regulated entity** willing to host the pilot described in section 12.

We are not asking for endorsement, funding, or regulatory approval of any product.

References (repository documents; available on request)

RBI Requirements Coverage matrix (requirement by requirement, with the test that meets each row or an explicit gap); RBI Agent Risk Map; BFSI Pilot Plan; Phase-1 daemon and transport specification; HSM custody note; backup and recovery note with its restore drill; operations runbook; Evaluation Package; Production Readiness; Sniffer Dog Complete Implementation Report; Reconciliation Implementation Report; Decision Point specification; Adversarial Results ledger (one row per attack, with raw output and the classification emitted) and the RED suite that reproduces it.